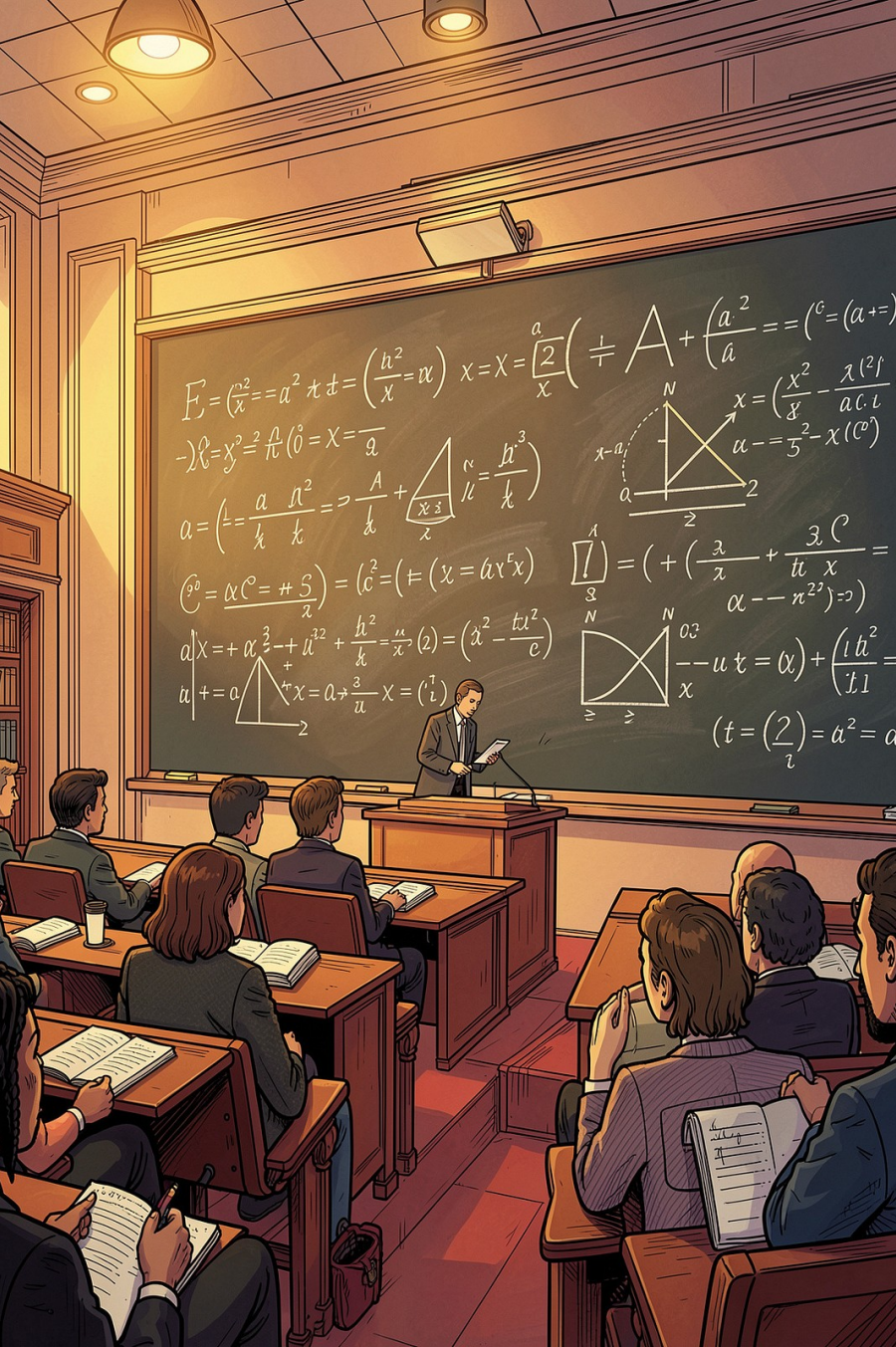




Polynomial-Time Reduction and NP-Completeness

Concepts, Examples, Applications and Modern Enhancements

- 📌 Audience: B.Tech CSE — concise, technically correct, classroom-ready



Why Study Computational Complexity?

Classify problems

Distinguish efficiently solvable problems from intractable ones.

Predict scaling

Understand how runtime grows with input size and impacts feasibility.

Guide algorithms

Select exact, approximate or heuristic methods for real problems.

Real-life example: Scheduling final exams — small timetables are easy; many constraints make the problem explode combinatorially.



Speaker note: Use the exam-scheduling example to motivate why classification matters.

Understanding Time Complexity

1

$O(n)$ — Linear

Time grows proportionally with input size.
Example: scanning an array.

2

$O(n^2)$, $O(n^3)$ — Polynomial

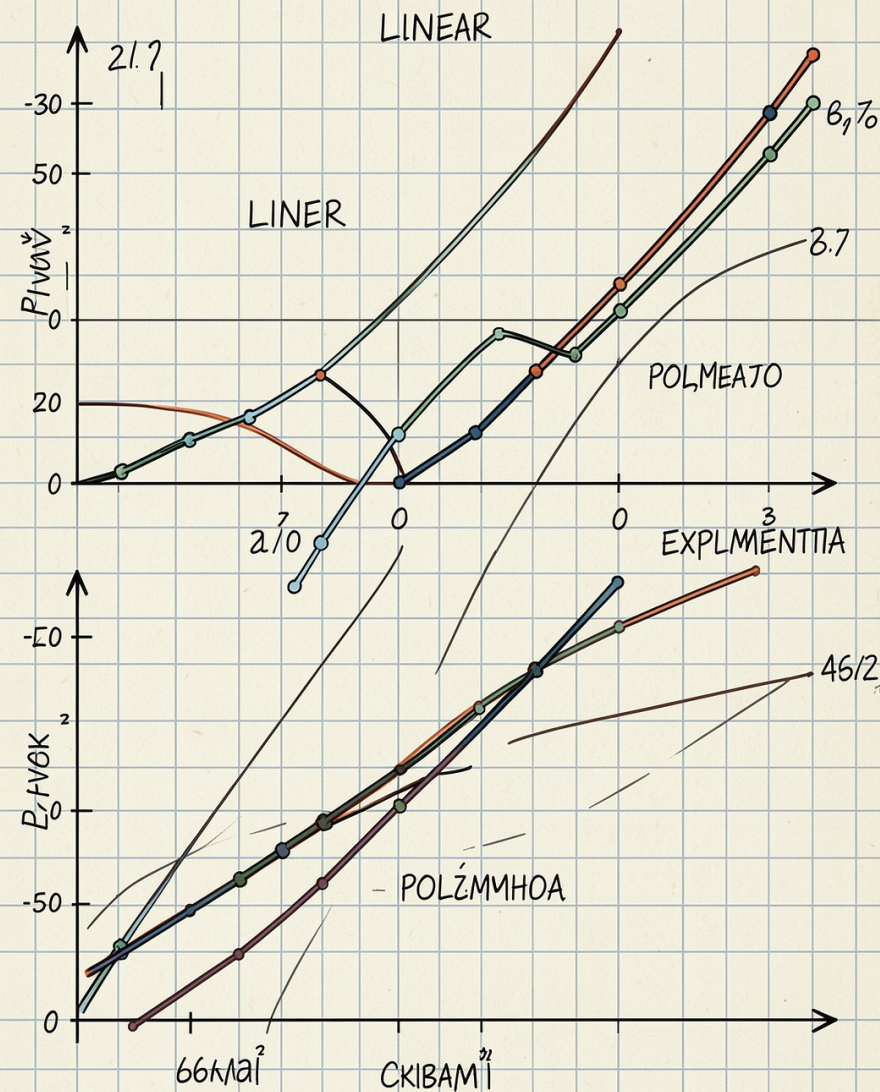
Often practical for moderate n ; common for many algorithms (sorting variants, graph algorithms).

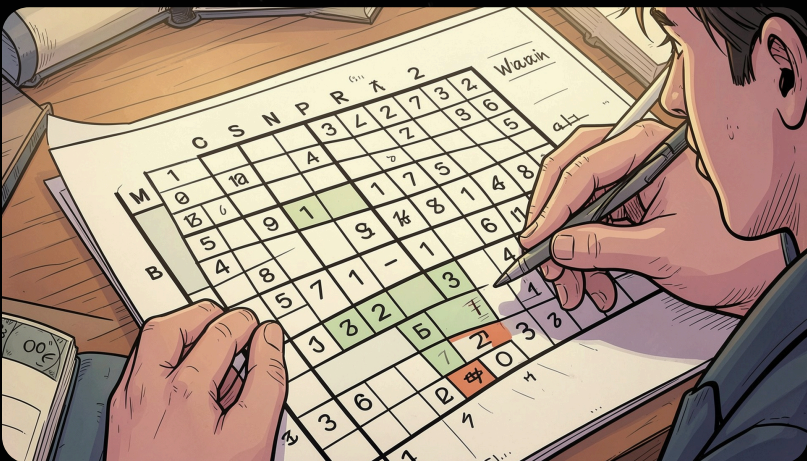
3

$O(2^n)$ — Exponential

Doubling input can square computation; often infeasible beyond small n .

Speaker note: Emphasise practical difference — polynomial usually manageable, exponential usually not.



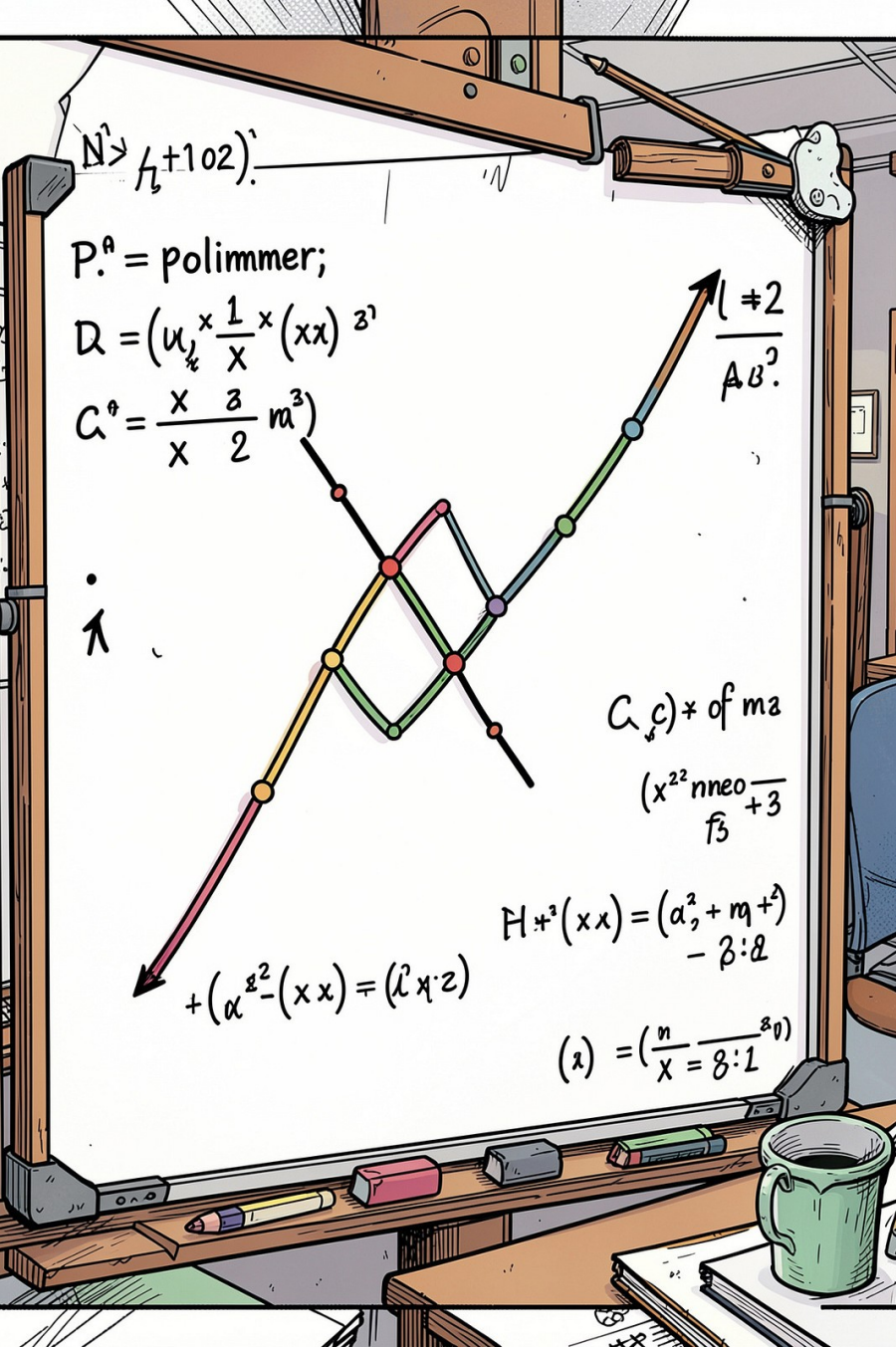


P and NP — Core Concepts

- **P:** Problems solvable in polynomial time (efficient algorithms exist).
- **NP:** Problems where a given solution can be verified in polynomial time.
- **Solving vs Verifying:** Solve might be hard; verifying a proposed solution is fast.

Example: Given a completed Sudoku, verifying correctness is quick (NP). Finding a solution from empty grid may be hard.

📌 Speaker note: Clarify that NP does not mean "not polynomial" — membership, not hardness.



What is Polynomial Time?

Polynomial time means runtime bounded by n^k for some constant k . Algorithms with polynomial runtime scale predictably and are considered feasible for practical use.

- Why considered feasible?

Polynomial growth keeps resource demands controllable for reasonable input sizes.

- Examples

- $O(n \log n)$ sorting
- $O(n^3)$ matrix multiplication (naïve)

📌 Speaker note: Emphasise constants and hidden factors matter in practice.

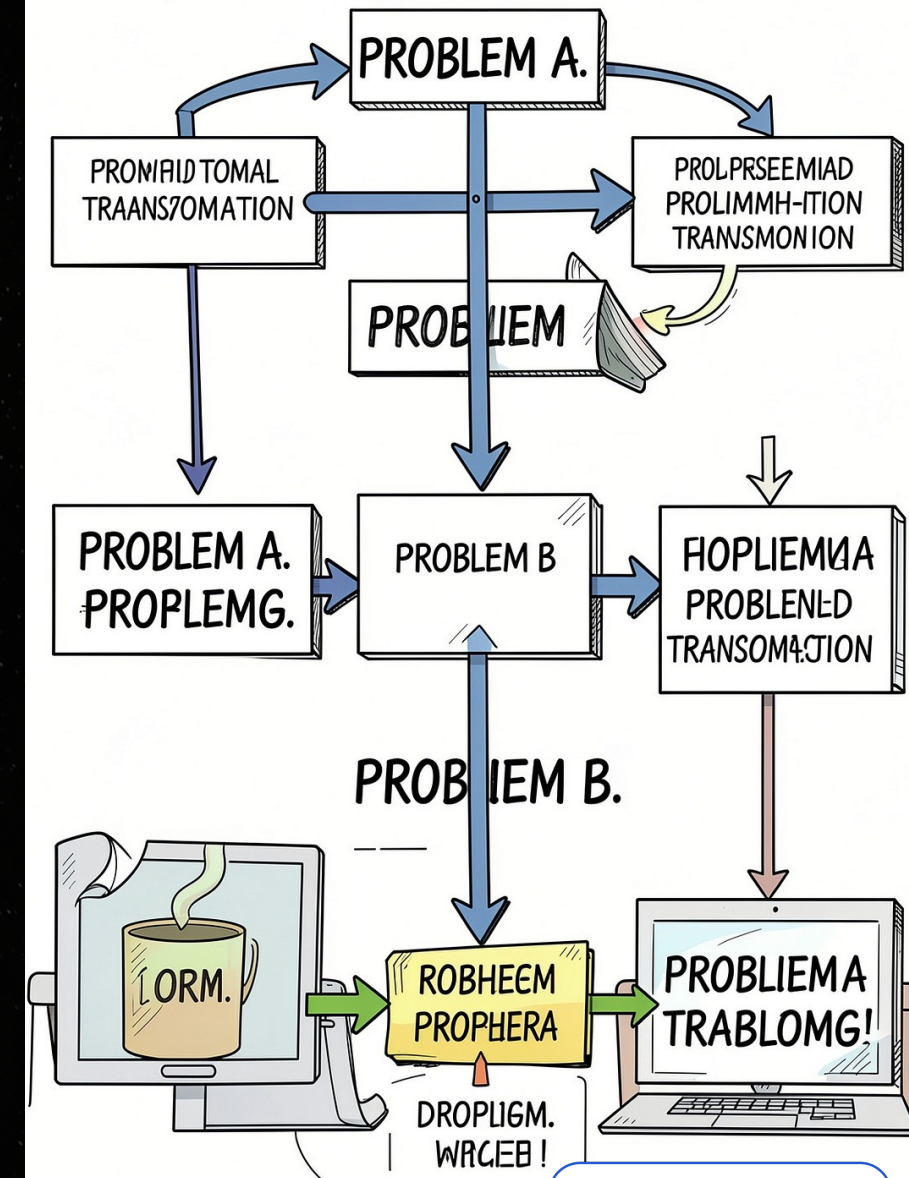
Polynomial-Time Reduction (Definition)

Reduction: A polynomial-time computable function f that maps instances of problem A to instances of problem B such that A-instance is yes iff $f(A)$ is yes for B.

Speaker note: Stress that reductions compare relative difficulty—if B is easy, and A reduces to B, then A is easy.

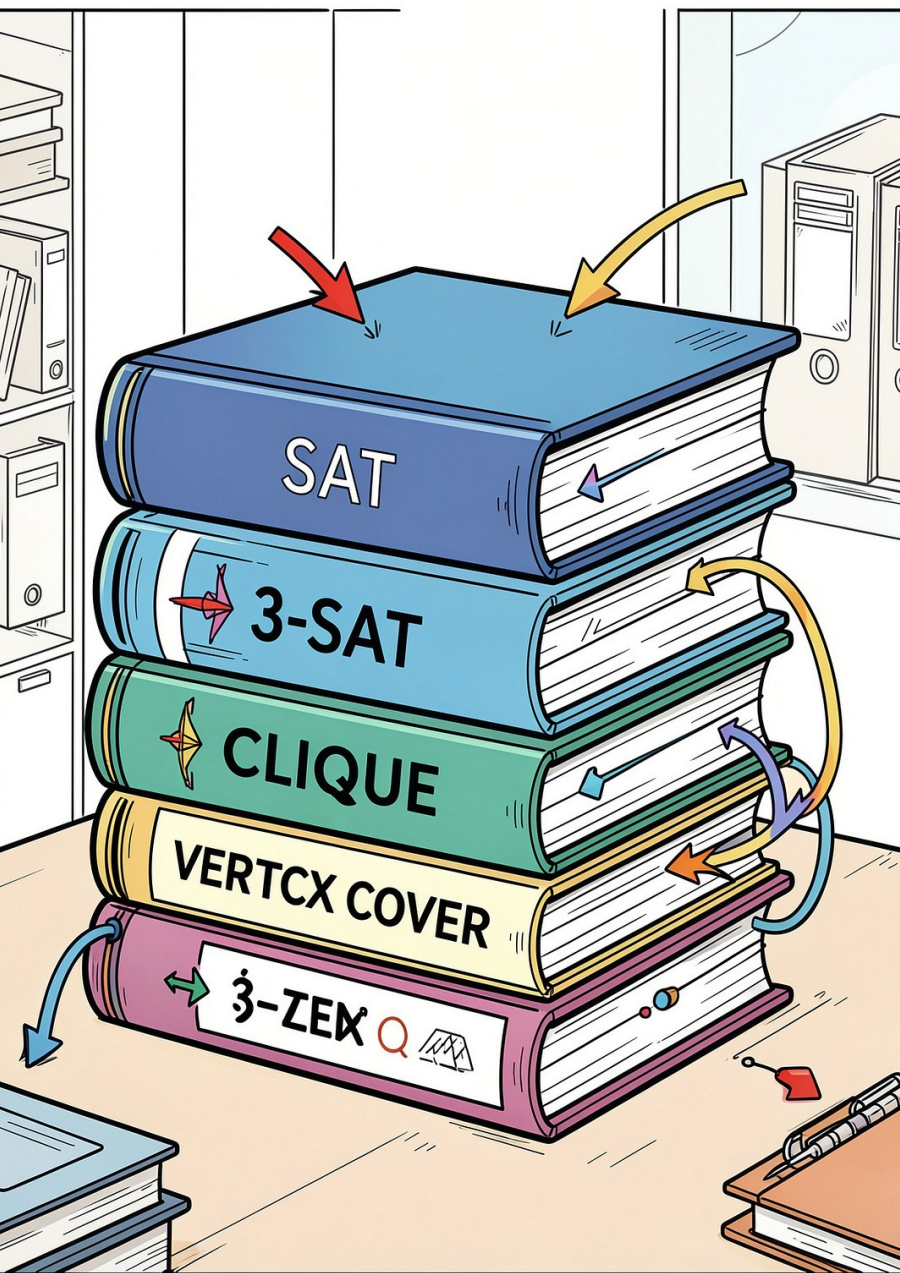


Flow-optic AlcRopime:



Made with **GAMMA**

STAIN



Why Reductions Matter

→ Prove hardness

Show a known hard problem reduces to a new problem to establish its difficulty.

→ Classify problems

Used to populate NP-Complete lists and guide algorithm choice.

→ Transfer algorithms

If B has efficient heuristics or approximations, A inherits practical methods via reduction.

📝 Speaker note: Use concrete reductions in class (e.g., 3-SAT → CLIQUE) to show technique.

Reduction Example: 3-SAT $\rightarrow$ CLIQUE (Sketch)

Step 1 — Start with 3-SAT

Formula: conjunction of clauses, each with 3 literals (e.g., $(x_1 \vee \neg x_2 \vee x_3)$).

Step 2 — Build graph

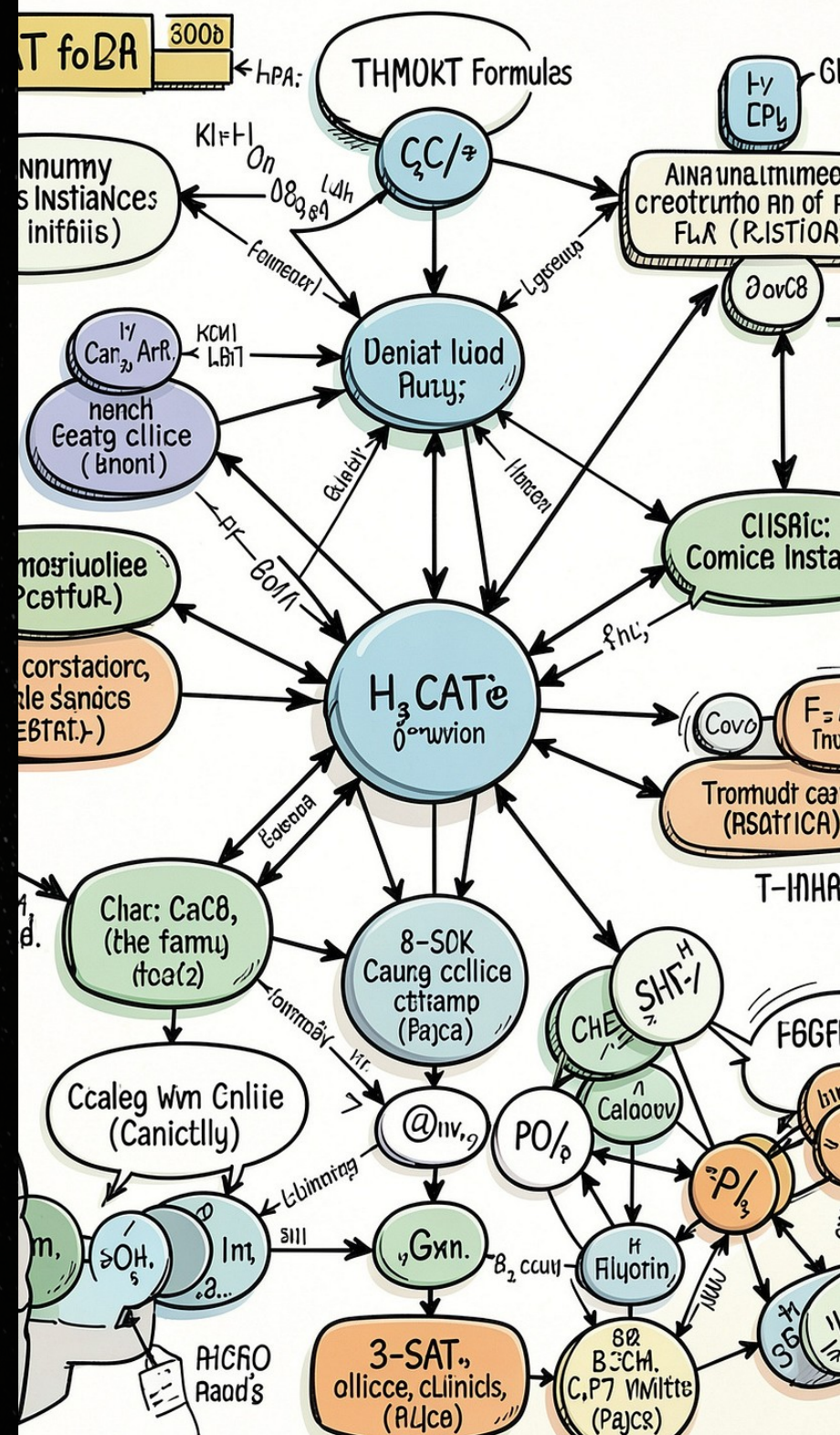
For each literal in each clause create a node. Connect nodes from different clauses unless they are contradictory (x and $\neg x$).

Step 3 — Clique target

Ask if graph has a k -clique where k = number of clauses. A k -clique corresponds to choosing one non-conflicting literal per clause that jointly satisfies the formula.

Key idea: Construction is polynomial in formula size; satisfiable $\Leftrightarrow$ clique of size k exists.

Speaker note: Present a tiny 2-clause example live to show node/edge mapping.



NP-Complete: Formal Criteria

1

1) In NP

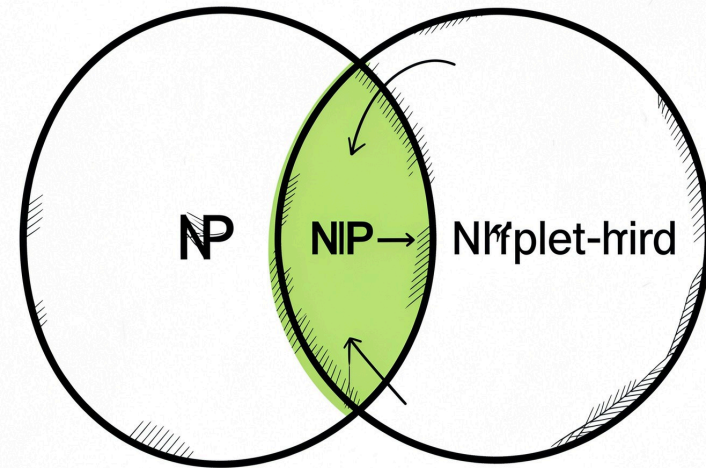
Solutions verifiable in polynomial time.

2

2) NP-hard

Every problem in NP reduces to it in polynomial time.

Conclusion: NP-Complete = $NP \cap NP\text{-hard}$.



Important: Whether $P = NP$ remains an open problem.



Speaker note: Do not assert $P \neq NP$; present open problem status clearly.

Modern Enhancements & Practical Handling



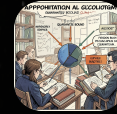
SAT Solvers

Highly optimized
CDCL solvers tackle
large SAT instances
efficiently in practice.



Heuristics & Metaheuristics

Greedy, local search,
simulated annealing
and genetic
algorithms yield good
solutions fast.



Approximation Algorithms

Provide provable
bounds when exact
polynomial-time
solution unlikely (e.g.,
TSP approximations).



AI/ML-assisted Optimization

ML models predict
promising regions of
the search space;
combined with
exact/heuristic
solvers.



Parallel & Hybrid Methods

Divide-and-conquer,
portfolio solvers, and
distributed search
improve practical
throughput.

Note: These techniques improve practical performance but do not imply polynomial-time solutions for NP-Complete problems.



Speaker note: Stress realistic expectations: better heuristics, not theoretical breakthroughs.